Executable Accountability for Verifier-Grounded Residual Ranking

Certificate-gated neural proposers under claim governance

Anonymous authors

Abstract. This paper studies executable accountability for verifier-grounded residual ranking. Exact or replayable checks own admissibility and consequences; small learned models rank, calibrate, reuse, or learn around the residual. The contribution is not that tiny networks solve hard tasks, but that verifier and replay artifacts become proposal structure while the manuscript is constrained by generated evidence. The clean anchor is `prime_gap`, where the generated claim row reports learned gate 68.87% vs hand-wheel 45.12% vs raw 9.13%; false rejections=0. The main feedback results are rejected-candidate reason learning and certified hypothesis-library promotion. Boundary rows for ARC, Signal, tiny-net capacity, and language-adjacent haiku probes test scope without expanding the central claim. The paper explicitly does not claim broad ARC competence, CDCL-equivalent learned clauses, comparative live neural-control improvement, semantic language understanding, or general end-to-end task solving.

1. Introduction

Machine-learning systems increasingly operate in settings where bad actions can be checked after the fact, but correctness should not be delegated to end-to-end model authority. Verifier-grounded systems should separate authority from ranking. Exact checkers, deterministic replay, and certificate tests should decide which candidates are admissible or what consequences a candidate produced. Learned components should spend their limited capacity on the residual: search order, calibration, reuse, and prioritization. The same discipline should govern the paper. Empirical prose should be downstream of trace rows, environment identity, checkpoint hashes, result summaries, and generated limitation candidates rather than manually maintained claims.

This paper is therefore a protocol paper, not an experiment catalog. The one-sentence thesis is:

"Verifier-grounded systems should let exact or replayable checks own admissibility, use small learned models only to rank the residual, and bind empirical claims to executable artifact evidence."

The operational summary is simple: the model proposes, the verifier decides, and the trace remembers enough to train the next proposal pass and govern the next manuscript claim.

Two empirical loops carry the result. First, rejected verifier rows are logged with reason codes and reused as supervision for a later safety-first residual ranker. Second, repeatedly certified hypotheses are promoted into a library and reused as proposal priors under held-out budgets. The prime-gap gate result supplies the clean anchor; the fixed-budget proposer failure supplies the falsifying negative row that motivates the feedback loop.

The remaining domains are boundary evidence. ARC tests grammar provenance and failure taxonomy, not benchmark competence. Signal tests replay and guarded live execution pressure, not comparative live-control performance. Tiny-net and adapter rows bound architecture claims. Haiku is language-adjacent corpus-proxy grounding, not semantic understanding. New domains stay outside the paper body until promoted through the evidence matrix.

The environment names are deliberately small. `prime_gap` tests exact arithmetic verification; `grid_path` tests topological path constraints; delayed-claim tests hidden outcome and safety-margin gates; ARC tests visual-spatial abstraction provenance; Signal tests bounded WASD control under replay and live telemetry; haiku tests corpus-derived proxy grounding.

At this artifact snapshot, the generated claim compiler reports 13 supported claims, 2 contradicted claims, 1 underpowered claim, 0 missing claims, and 0 stale claims. Contradicted and underpowered rows remain visible because falsifiability is part of the protocol. The fixed-budget proposer row is contradicted (static neural proposer trails deterministic baselines on at least one grammar; candidate static=72.49 saved vs grammar=73.22, conditional static=2.15 saved vs grammar=55.57.), and the tiny-net plateau row is underpowered (tiny-net plateau audit covers 4 capacities, 4 domains, and variants shared, shared_task_heads, single_task; 15/48 required rows meet the plateau threshold and 1/12 domain/variant verdicts are audit-ready.).

2. Positioning

The formal-methods ancestry is counterexample-guided search. Sketching combines inductive synthesis with validation-generated counterexamples [1], and SyGuS names the grammar-plus-specification setting [2]. CEGAR and CDCL add the adjacent lesson that failed traces can refine later search rather than disappear as waste [3], [4], [5]. This paper inherits that shape but uses failures as supervised ranking data, not as sound learned clauses.

Modern verifier-bounded learning asks models to solve or help verify reasoning traces [6], [7], [8], [9], [10]. AlphaGeometry-style systems combine learned proposal with formal checking [11], [12], and theorem-proving work similarly uses neural proposal around formal environments [13]. The artifact here is smaller and narrower: the learned model never owns correctness.

The accountability lineage is equally important. Model Cards, Datasheets, reproducibility checklists, and reinforcement-learning reproducibility critiques made documentation and artifact discipline central research objects [14], [15], [16], [17]. The claim compiler is an executable version of that idea: claims, numbers, limitations, and support status are generated from repository artifacts before the paper compiles.

3. Protocol

Every promoted row has the same contract:

```
Task T = {
  state distribution S_T,
  candidate enumerator A_T(s),
  observable features x_T(s, a),
  residual scorer f_theta(x_T),
  grounding operator O_T(s, a),
  grounding authority level,
  grammar G_T,
  certificate or replay checker C_T,
  trace / replay / claim protocol
}
```

The invariant is:

```
The model proposes. The grounding operator decides. The trace remembers.
```

Stage	Concrete prime-gap object	Artifact that survives
Candidate	The proposer emits a modular gate candidate such as "reject candidate `n` when `n > 3` and `n % 3 == 0`" for next-prime search states.	Grammar id, candidate id, split, seed, and support window.
Gate check	The segmented-sieve verifier checks whether the rule removes any true next-prime outcome under the declared support.	A false-rejection certificate; accepted only when false rejections are zero.
Residual rank	Certified composite candidates are removed, and the scorer ranks the remaining action ids by residual score.	Top-k metrics, stable action ids, ranking policy, and score margins.
Trace emit	The run writes comparison, rule-search, and certificate-trace rows.	`crlp.compare.v1`, `crlp.rule_search.summary.v1`, environment and model hashes.
Claim consume	The claim compiler derives the paper macro from the artifact summary.	`claim_C1_prime_gate_delta_summary` and generated support/limitation status.

Table 1: Concrete prime-gap trace. The paper number is downstream of candidate generation, verifier certification, residual ranking, trace emission, and generated claim consumption.

An exact verifier is the strongest grounding operator, but the protocol also admits deterministic replay and explicitly weaker corpus proxies. Authority level determines claim scope.

Authority	Operator	Examples	Allowed claim
exact check	proof, lookup, type check, deterministic verifier	`prime_gap`, `grid_path`, certified ARC transform rows	correctness and zero-false-rejection claims inside declared support
deterministic replay	simulator or replay deciding candidate consequences	Signal flight, delayed-claim simulator	replay-certified utility, regret, damage, death, or decision metrics
corpus proxy	train-split grammar, adjacency, corruption benchmark	haiku prototypes, public corpus coverage surfaces	corpus fit and near-miss discrimination, not semantic truth
human review	reviewable acceptance/rejection	editorial-agent pull requests	review-gated assistance, not autonomous empirical authority

Table 2: Grounding authority levels. The same residual scorer has different evidentiary force depending on which operator produced the outcome row.

The task-independent rules are concise. Candidate enumeration is complete before gating. Features declare provenance, and grounding-derived quantities are labels, rewards, diagnostics, or gates unless the schema explicitly permits them as inputs. Rules enter the gate only through the task grammar and certificate or replay checker. JSONL traces use stable candidate ids, ranking policy, score-margin metadata, environment identity, and replay semantic hashes so reordered candidates or float ties cannot silently change the paper's meaning.

4. Evidence Matrix

The matrix is the scope boundary. A row may be interpreted in the prose only if its data, candidate grammar, grounding operator, scorer role, trace artifact, result, claim, and limitation are registered. The paper may tighten wording without adding experiment families; new families require revising this matrix first.

Family	Grounding	Learned Role	Claim / Boundary
prime gap	**Grounding:** segmented-sieve replay and modular certificate checks zero-false-rejection divisibility certificates over train/validation support	rank residual gaps and budgeted certified rules	**Claim:** certified modular gates shrink the residual action space. **Limit:** static neural proposer is not superior to deterministic baselines at fixed budget
grid path	**Grounding:** BFS shortest-path successor and rule certificate checks legality, trap, clear-path, and distance-dominance certificates	rank legal residual moves after symbolic constraints	**Claim:** same gate contract works outside residue arithmetic. **Limit:** supporting transfer surface, not a main contribution
delayed claim	**Grounding:** deterministic hidden-outcome simulator and rule certificate checks finite-generator safety check plus false-rejection counters	rank utility only after safety dominates the objective	**Claim:** reason learning needs domain-aware safety constraints. **Limit:** reason-prior transfer alone can fail safety; claim remains domain-scoped
rejected candidate learning	**Grounding:** original rule/certificate checker that rejected or accepted each row false-rejection-first budget filter over reason priors and residual scores	learn residual ordering from rejection reasons and marginal verifier utility	**Claim:** verifier failures become training signal. **Limit:** not a generic reason-code ranker; delayed-claim safety remains scoped
hypothesis library feedback	**Grounding:** train-pair exactness, type checks, and held-out fixture split evaluation zero-false-rejection promotion before a binding enters the library	rank promoted slot priors and residual program candidates under tight budgets	**Claim:** certified learning can accumulate through reusable library priors. **Limit:** public ARC promotion remains scoped evidence, not a headline ARC score
multitask tiny net	**Grounding:** task-local labels and certified downstream gates per-task held-out metrics, OOD ARC variants, calibration bins	shared residual scorer with domain-specific diagnostics	**Claim:** representation/protocol limits dominate naive width scaling. **Limit:** architecture-size claims stay gated by the plateau audit
arc agi evidence surface	**Grounding:** train-pair consistency, unique compiled prediction, and public corpus loader program/hypothesis certificate rows and scorer-invariant loader summaries	rank bounded candidates; scorer does not own correctness	**Claim:** ARC exposes priors, grammar coverage, and failure taxonomy. **Limit:** not a broad ARC competence or benchmark-score claim
signal flight	**Grounding:** offline counterfactual simulator replay, closed-loop rollout replay, live shadow frame comparison, and live active telemetry replay utility, regret, damage/death counters, certified control-target rollout rows, shadow disagreement logs, live safety gates, and protocol compatibility rows	rank bounded control candidates beside heuristic control	**Claim:** application pressure for replay-grounded residual ranking and guarded live neural-active execution. **Limit:** active neural versus heuristic performance is not yet claimed

Table 3: Frozen evidence matrix. Rows use a four-column protocol view to avoid treating the paper as a leaderboard or a catalog.

5. Prime-Gap Anchor

The prime-gap row is the cleanest evidence because the verifier is exact and the gate has a clear safety property. Candidate actions are possible prime gaps after a segmented-sieve window. Candidate gates are modular rejection predicates. A gate is accepted only if its certificate has zero false rejections over the declared support; the scorer then ranks the residual candidates. The `hand wheel` baseline is a deterministic hand-coded wheel-factorization filter over small modular divisibility rules.

Condition	Top-1	Top-10	False rejections
no gate	9.13%	78.01%	0
hand wheel	45.12%	99.77%	0
learned wheel	**68.87%**	**100%**	**0**

Table 4: Prime-gap certified modular gate on the canonical sample of 30000 rows. learned gate 68.87% vs hand-wheel 45.12% vs raw 9.13%; false rejections=0. Learned-vs-hand top-1 delta: +23.74 pp.

The seed audit closes the headline-stability gap for this anchor: 3 seeds: learned 68.87% +/- 0.00pp vs hand 45.12% +/- 0.00pp and raw 9.13% +/- 0.00pp; min learned-vs-hand delta +23.74pp, learned top-10 min 100.00%, false rejections max=0. The zero spread is a seed-stability audit over this deterministic benchmark: seeds vary checkpoints and proposer runs, not the declared prime-range evaluation distribution.

This result is intentionally narrow. It supports the protocol claim that certified structure can shrink residual search sharply while preserving correctness under the declared support. It does not imply that a static neural proposer is better than deterministic symbolic ordering. The generated negative row says the opposite: static neural proposer trails deterministic baselines on at least one grammar; candidate static=72.49 saved vs grammar=73.22, conditional static=2.15 saved vs grammar=55.57. That failure is useful because it prevents the paper from becoming a generic "neural proposal wins" story.

The naive learned proposer fails. That is the point. Unsupported neural proposal is not trusted; verifier feedback is recycled until proposal economics improve. In this protocol, negative rows are not cleanup work after the fact. They are the training substrate that separates residual ranking from admissibility.

6. Feedback Loops

The main contribution is what happens after verifier feedback exists.

Rejected-candidate learning. Failed rule proposals are logged with task, split, seed, reason family, false-rejection counts, and marginal verifier utility. Those rows become targets for a later residual ranker. The supported claim is scoped to prime/modular sweeps: reason learning repairs the prime proposer failure at budget 8: model_score selects 7 unsafe rows and causes 172 validation false rejections, while reason_prior reaches 6 accepted rules/216.0 utility and trained_residual reaches 7 accepted rules/245.0 utility with zero false rejections; prime/modular split sweeps pass, but delayed_claim has 0 mixed split rows.

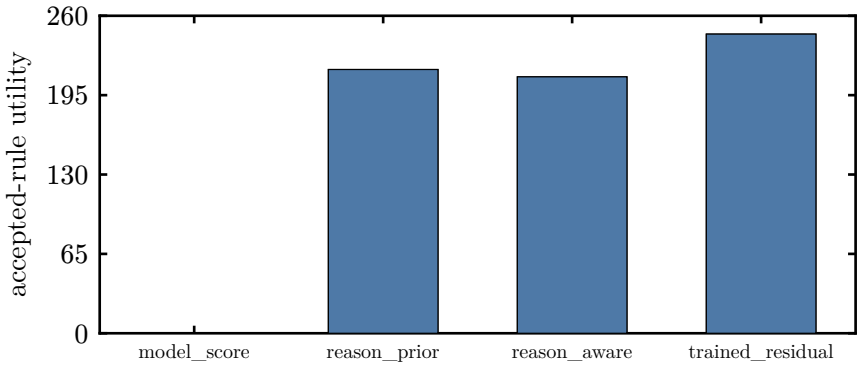


Figure 1: Rejected-candidate reason learning. Verifier failures become training signal without making the learned model authoritative.

Certified hypothesis-library feedback. Repeated zero-false-rejection hypotheses are promoted into a reusable library, converted into slot priors, and checked on held-out tasks under the same tight budget. The supported claim is: certified-library promotion supports fixture-level feedback: promoted move_to_anchor from 2 tasks/78 covered events with zero false rejections; slot priors raise learned certified@1 from 2 to 4 and utility@8 from 0.2119 to 0.5169. Held-out split feedback rows: 30.

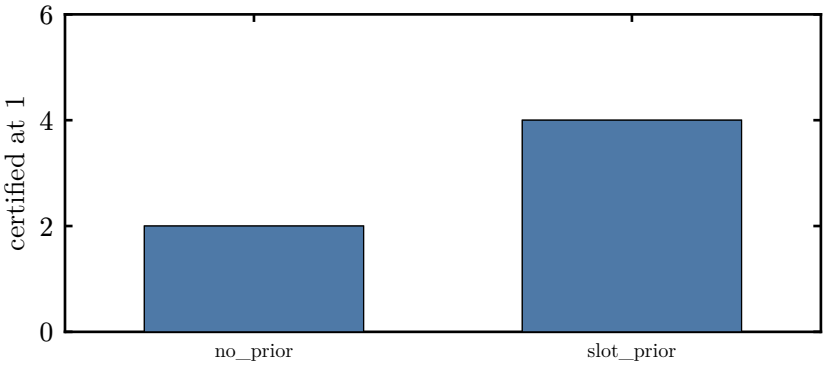


Figure 2: Certified hypothesis-library promotion. Certified hypotheses become proposal priors only after passing the promotion gate.

Together these rows are the paper's feedback mechanism. Rejections and certifications are not post-hoc diagnostics; they are reusable artifacts that change the next proposal pass.

7. Boundary Evidence

Boundary rows keep the thesis honest without becoming separate stories.

ARC. ARC is a provenance and grammar-coverage stress surface. The paper reports the current measured public-corpus surface, not a headline ARC score: public ARC evidence surface improves from v10 eval 4 tasks/187 cells to v15_measured eval 7 tasks/1057 cells; v15_measured train remains 24 tasks/1187 cells. Calibration remains conditional: ARC shared scorer is a calibration tradeoff: fixed-task NLL improves (4.0827 vs 8.2754) while ECE worsens (0.6995 vs 0.5589); OOD improves both NLL (2.6278 vs 4.3723) and ECE (0.4481 vs 0.4820).

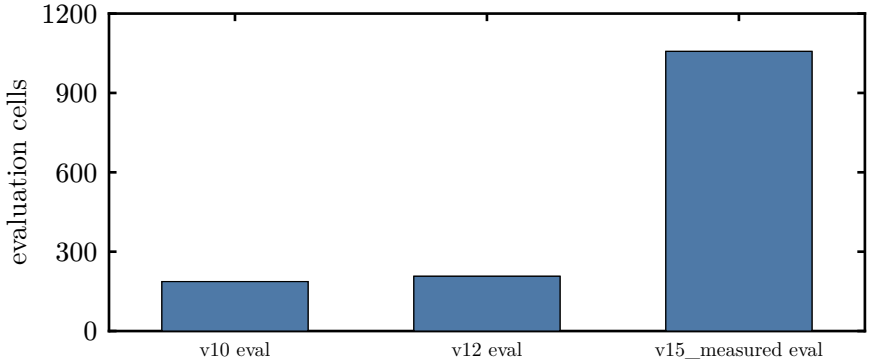


Figure 3: Public ARC evidence surface. This is measured `v15` coverage and failure taxonomy, not broad ARC competence.

Signal. Signal is a deterministic-replay pressure test. Offline replay scores bounded WASD candidate actions against recorded simulator consequences, and closed-loop replay evaluates guarded rollout rows before any live-control language is admitted. The live `neural_active` row is narrower: a running local Signal pilot applies checkpoint-ranked WASD proposals only through protocol checks and safety gates, and those gates may veto execution instead of letting the neural top choice own control.

The trace records active source, checkpoint hash, latency, protocol compatibility, safety-gate rows, damage/death telemetry, and replay-ready frames: Signal offline replay ranker top-1 79.76% vs baseline 67.86% (+11.90pp), utility 0.1432 vs 0.1359, regret 0.0073 vs 0.0146, damage 2.38% vs 2.38%, deaths 0.00% vs 0.00%; live shadow frames=74. Closed-loop rollout neural_guarded utility 0.3564 vs baseline -1.4642, reach 12.50% vs 8.33%. Signal live neural-active pilot executed 74 recent neural-source rows from checkpoint f12b644ed9fa with p95 inference 0.089 ms, 74 replay-ready rows, protocol-compatible rows=74, safety-gate rows=74, damage events=2, deaths=0; no baseline comparison is claimed. The paper still does not claim active neural control improves utility, regret, damage, or death versus the heuristic baseline.

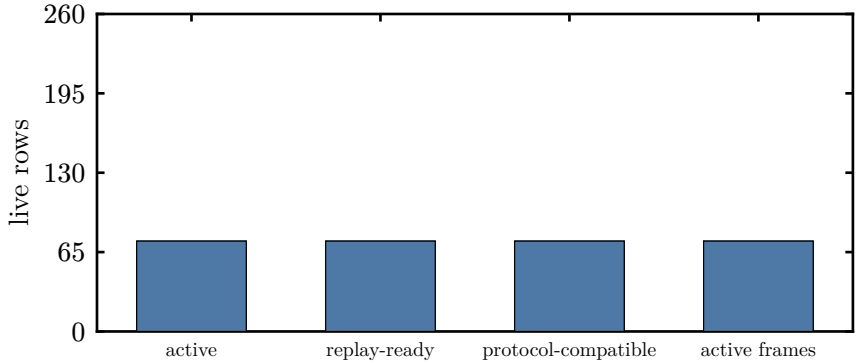


Figure 4: Signal guarded live neural-active execution. The row establishes live checkpoint-ranked execution and replay-ready telemetry, not comparative active-control superiority.

Architecture and language-adjacent probes. Shared-scorer, adapter, and tiny-net rows remain boundary evidence: capacity sweep rules out a simple wider-is-better ARC story: best fixed-ARC shared row is 32x16 at 39.20%, but every shared ARC row trails single-task; delayed_claim deltas remain +0.10pp to +0.43pp. adapter-head audit: fixed ARC heads-flat delta +2.40pp; delayed_claim heads-flat -0.97pp; grid_path heads-flat +0.07pp; ARC OOD heads-single +4.67pp top-1 and -1.1836 NLL. Haiku prototypes use corpus-derived vocabulary, adjacency, syllable/shape constraints, and real-vs-corrupt discrimination as a `corpus_proxy` operator. They test how the architecture behaves when the "verifier" is only dataset grounding; they are intentionally not promoted as semantic language understanding.

8. Claim Governance

The paper construction layer mirrors the protocol. Result scripts emit canonical summaries, the claim compiler converts those summaries into Typst macros and limitation candidates, and the PDF consumes those macros rather than hand-maintained numbers. The trace determinism row is currently supported: trace determinism audit covers 2 traces and 2000 rows; invalid_rows=0, rank_fp_sensitive_rows=0, logged_mismatches=0, semantic_hashes_equal=True. This is not a claim that every sentence is cryptographically enforced. It is the narrower claim that registered empirical claims cannot remain supported when their required artifacts are missing, stale, contradicted, or underpowered.

Class	Count	Body rule
supported	13	May be interpreted, using generated summaries and artifact-backed scope.
contradicted	2	Must remain negative evidence, limitation, or claim-change work.
underpowered	1	May motivate roadmap items, not body-strength architecture claims.
missing/stale	0 / 0	Must block or remove body claims until artifacts are regenerated.

Table 5: Generated claim-gate summary. The detailed support-surface rows remain in `paper/generated/claims.typ`; the body uses the counts and generated claim macros rather than forcing every long row into the PDF.

Editorial agents fit the same governance model. They may improve wording, structure, transitions, and clarity through reviewable pull requests. They may not change claim scope, numeric results, generated summaries, citations, or experiment interpretation unless they open an explicit claim-change PR. This disclosure is operational rather than decorative: AI agents are reviewable assistants, while empirical claims are generated from repository artifacts and checked before compilation.

9. Limitations

1. **Scope is deliberately narrow.** The paper supports verifier-grounded residual ranking and executable claim governance. It does not support broad task solving, broad ARC competence, semantic language understanding, or competitive theorem proving.
2. **Multi-seed intervals are still uneven.** The clean prime-gate headline now has a generated seed interval. Some boundary rows still need more explicit confidence intervals before they should be treated as stable quantitative laws.
3. **The fixed-budget proposer result is negative.** The generated contradiction is not an embarrassment; it is the reason rejected-candidate learning matters.
4. **Architecture claims are underpowered or contradicted.** Tiny-net plateau evidence remains underpowered, and adapter heads are diagnostic rather than promoted architecture.
5. **Signal is live but not comparative.** The live `neural_active` row establishes guarded checkpoint-ranked execution with replay-ready telemetry. It does not establish active neural improvement over the heuristic baseline.
6. **Corpus proxies are weaker than verifiers.** The haiku task can ground train-split adjacency, shape, and corruption discrimination. It cannot ground truth or semantics.
7. **Executable accountability is partial.** Claim compilation catches unsupported generated claims, stale evidence, and limitation candidates. It does not replace human judgment about interpretation, ethics, or whether a domain belongs in the paper.

Claim	Status	Generated limitation candidate
C2.prime_proposer_fixed_budget	contradicted	static neural proposer trails deterministic baselines on at least one grammar; candidate static=72.49 saved vs grammar=73.22, conditional static=2.15 saved vs grammar=55.57.
C4.adapter_heads_shared_trunk	contradicted	adapter-head audit: fixed ARC heads-flat delta +2.40pp; delayed_claim heads-flat -0.97pp; grid_path heads-flat +0.07pp; ARC OOD heads-single +4.67pp top-1 and -1.1836 NLL.
C7.tiny_net_plateau_stop_rule	underpowered	tiny-net plateau audit covers 4 capacities, 4 domains, and variants shared, shared_task_heads, single_task; 15/48 required rows meet the plateau threshold and 1/12 domain/variant verdicts are audit-ready.

Table 6: Generated limitation candidates from the paper-claim compiler. The required claim support gate is currently passing. Optional contradicted claims remain visible here instead of blocking PDF compilation.

10. Conclusion

The result is a disciplined loop: enumerate candidates, ground them with an exact check or replayable operator, let a small model rank the residual, preserve the trace, and bind the paper's claims to generated evidence. Prime-gap shows the clean certified-gate anchor. Rejected-candidate learning and hypothesis-library promotion show that verifier artifacts can feed back into later proposal structure. ARC, Signal, tiny-net, adapter, and haiku rows mark the boundary rather than widening the thesis.

The broader lesson is pragmatic. When correctness, safety, or auditability matter, the model should not own the whole decision and the paper should not own the whole narrative. The checker or replay system owns admissibility and consequence. The learned component owns ranking pressure. The repository trace owns what the manuscript is allowed to say.

Bibliography

[1] A. Solar Lezama, "Program Synthesis By Sketching," Doctoral dissertation, 2008. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-177.html

[2] R. Alur *et al.*, "Syntax-Guided Synthesis," in *Proceedings of the IEEE International Conference on Formal Methods in Computer-Aided Design (FMCAD)*, 2013, pp. 1–17.

[3] E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-Guided Abstraction Refinement," in *Computer Aided Verification (CAV)*, 2000, pp. 154–169. doi: 10.1007/10722167_15.

[4] J. P. Marques-Silva and K. A. Sakallah, "GRASP: A Search Algorithm for Propositional Satisfiability," *IEEE Transactions on Computers*, vol. 48, no. 5, pp. 506–521, 1999.

[5] R. J. Bayardo and R. Schrag, "Using CSP Look-Back Techniques to Solve Real-World SAT Instances," in *Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI)*, 1997, pp. 203–208.

[6] K. Cobbe *et al.*, "Training Verifiers to Solve Math Word Problems," *arXiv preprint arXiv:2110.14168*, 2021.

[7] H. Lightman *et al.*, "Let's Verify Step by Step," *arXiv preprint arXiv:2305.20050*, 2023.

[8] J. Uesato *et al.*, "Solving math word problems with process- and outcome-based feedback," *arXiv preprint arXiv:2211.14275*, 2022.

[9] A. Hosseini, X. Yuan, N. Malkin, A. Courville, A. Sordoni, and R. Agarwal, "V-STaR: Training Verifiers for Self-Taught Reasoners," *arXiv preprint arXiv:2402.06457*, 2024.

[10] DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," *arXiv preprint arXiv:2501.12948*, 2025.

[11] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong, "Solving olympiad geometry without human demonstrations," *Nature*, vol. 625, pp. 476–482, 2024.

[12] Y. Chervonyi *et al.*, "Gold-medalist Performance in Solving Olympiad Geometry with AlphaGeometry2," *arXiv preprint arXiv:2502.03544*, 2025.

[13] S. Polu and I. Sutskever, "Generative Language Modeling for Automated Theorem Proving," *arXiv preprint arXiv:2009.03393*, 2020.

[14] M. Mitchell *et al.*, "Model Cards for Model Reporting," in *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, 2019, pp. 220–229. doi: 10.1145/3287560.3287596.

[15] T. Gebru *et al.*, "Datasheets for Datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021, doi: 10.1145/3458723.

[16] J. Pineau *et al.*, "Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

[17] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep Reinforcement Learning That Matters," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, 2018, pp. 3207–3214. doi: 10.1609/AAAI.V32I1.11694.